

FlexFit Studio — Refactor Documentation

This document explains the architectural changes made in Project 1 (FlexFit Studio), why each change was made, and how application behavior was preserved.

Goal

Restructure a working gym management app into a codebase that follows modern Next.js and TypeScript practice, without changing what the app does. Every feature that worked before must still work with the same inputs, outputs, errors, and edge cases.

Summary of Changes

Every single router in the application has been refactored into a thin adapter layer. All database queries, transaction orchestrations, and business checks are now located in isolated, testable service modules.

Area	Before	After
tRPC Setup	Single <code>trpc.ts</code> file	Separated into <code>server/trpc/index.ts</code> with a slim <code>trpc.ts</code> export shim to prevent breaking existing imports.
Duplicated Helpers	<code>hoursUntil</code> and <code>activeMembershipFor</code> duplicated across 3 separate files. Policy constants scattered inline.	Extracted into shared libraries under <code>server/lib/</code> (<code>time.ts</code> , <code>memberships.ts</code> , and <code>constants.ts</code>).
Booking Operations	<code>bookings.ts</code> (~405 lines) handled lists, booking, cancellations, rosters, and waitlists.	Decoupled into <code>server/services/member-bookings.ts</code> . Router is now ~60 lines.
Rescheduling Flow	<code>reschedules.ts</code> (~380 lines) duplicated validation logic across queries and mutations.	Extracted pure, database-free validation rules to <code>server/lib/reschedule-validation.ts</code> . Extracted mutation orchestrations to <code>server/services/reschedules.ts</code> . Router is now ~45 lines.

Corporate Bookings	<code>corporate-bookings.ts</code> (~325 lines) mixed validation, billing, and credit check logic.	Moved to <code>server/services/corporate-bookings.ts</code> . Router is now ~55 lines.
Admin Reports	<code>admin.ts</code> (~270 lines) contained heavy SQLite/Drizzle queries for statistics, revenue, no-shows, etc.	Extracted reporting queries to <code>server/services/admin.ts</code> . Router is now thin endpoints.
Class Management	<code>classes.ts</code> (~155 lines) contained mutation validations, lists, and class cancellations with side effects.	Decoupled database actions to <code>server/services/classes.ts</code> . Router delegates cleanly.
Trainer Availability	<code>trainers.ts</code> (~210 lines) performed custom availability window overlap calculations and conflict checks inline.	Extracted checks and database modifications to <code>server/services/trainers.ts</code> . Router is purely wire-routing.
Member Profiles & Search	<code>members.ts</code> (~165 lines) handled profile retrieval, name updates, staff queries, and directory lookups.	Moved database operations to <code>server/services/members.ts</code> . Router only validates arguments.
Company Administration	<code>admin-companies.ts</code> (~225 lines) managed member-to-company linkage, top-ups, and booking summaries.	Encapsulated logic inside <code>server/services/admin-companies.ts</code> . Router is thin.
Subscription Plans	<code>plans.ts</code> (~95 lines) handled subscription purchases, date additions, and payment records inline.	Extracted purchase steps and active-state changes to <code>server/services/plans.ts</code> . Router is thin.
Payment Audits & Refunds	<code>payments.ts</code> (~110 lines) handled refund validations and cascading cancellations of memberships inline.	Decoupled steps into <code>server/services/payments.ts</code> . Router delegates cleanly.
System Notifications	<code>notifications.ts</code> (~75 lines) handled unread counts, list fetching, and admin broadcasts.	Decoupled notification state triggers into <code>server/services/notifications.ts</code> .
Authentication Flow	<code>auth.ts</code> (~110 lines) handled cryptographical hashing, user creation, and Next.js cookie manipulations.	Split concerns: session insertions, hashing, and lookups moved to <code>server/services/auth.ts</code> , while HTTP-specific cookie stores remain in the router.

Pages & UI Views	Dashboard and trainer pages mixed API calls, state handlers, and large inline JSX views.	Cleanly divided views: created page-specific presentation components in <code>components/features/</code> and let pages act as the fetch controllers.
Tests	Zero tests (vitest configured but unused).	14 unit tests under <code>src/server/lib/business-rules.test.ts</code> covering core policies, boundary dates, and rescheduling.

New Folder Structure

The refactored project structure is organized as follows:

```
src/ app/ # Next.js routes and pages (unchanged URLs) components/ features/ dashboard/ # Member
dashboard layout UI sections trainer/ # Trainer schedule UI sections NavBar.tsx # Shared
navigation reschedule-modal.tsx # Shared modal db/ # SQLite schema definitions & clients
(unmodified) lib/ # Client-side utility functions (unmodified) server/ lib/ # Shared server-side
helper modules class-validation.ts # Central assertClassBookable guard constants.ts # Policy
variables (12h cancel, 4h reschedule, etc.) db.ts # Shared database type alias memberships.ts #
Active membership checks reschedule-validation.ts # Pure reschedule logic validator
(database-free) time.ts # Date and time helpers business-rules.test.ts # Unit tests for policy
calculations services/ # Core Business Logic Layer (isolated from trpc) admin.ts # Reporting
analytics & stats queries admin-companies.ts # Corporate account and member link operations
auth.ts # Login, register, and session management classes.ts # Class scheduling, updates, and
cancellations corporate-bookings.ts # Company credit-pool reservation lifecycles
member-bookings.ts # Personal class reservation lifecycles members.ts # Profile updates,
lookups, and staff search queries notifications.ts # Read counts, listing, and broadcast alerts
payments.ts # Payment logs, approvals, and refund reversals plans.ts # Subscription plan checkout
triggers reschedules.ts # Reschedule operations and audit histories trainers.ts # Trainer
availability and conflict checking trpc/ index.ts # Procedure contexts, authorization
middlewares trpc.ts # Backward-compatible router shim routers/ # Thin trpc Router Adapters (only
handle Zod input)
```

Why This Architecture Was Chosen

1. Separation of Concerns (MVC / Clean Architecture principles):

- **Routers** (`server/routers/`): Act purely as HTTP controllers/adapters. Their only responsibility is to parse and validate incoming inputs using Zod schemas and then hand off execution. They are free from SQL queries and transactional logic.
- **Services** (`server/services/`): Act as the business domain layer. They handle database operations (Drizzle transactions, inserts, updates) and execute domain operations.

- **Utilities/Helpers** (`server/lib/`): Act as pure utilities or state validation guards. Putting constants and boundary checks here allows sharing them easily (e.g. sharing cancellation windows across members and corporate members) and testing them in isolation.

2. Policy Synchronization (One Source of Truth):

By centralizing cancellation limits (e.g. 12 hours for members, 24 hours for corporate) and unlimited credit thresholds (e.g. `999`) in `server/lib/constants.ts`, we avoid policy drift.

3. High Testability:

Core logic (like reschedule rules, date math, or cancellations) can be verified via pure tests in `business-rules.test.ts` without spinning up local database engines, keeping tests extremely fast.

4. Maintainability and Developer Onboarding:

Breaking up large, monolithic router files (some of which exceeded 400 lines and performed 5 unrelated checks) into separate services with singular responsibilities makes code navigation faster and debugging intuitive.

Retained Quirks and Edge Cases

Every behavioral quirk in the original codebase has been retained verbatim to preserve matching inputs/outputs:

1. **Waitlist promotions on cancel:** Confirmed cancels promote the longest-waiting member. No credits are charged until the waitlisted member is promoted.
2. **Corporate Kiosk Check-ins:** Front-desk kiosk check-ins record `bookingId: null` in the checkins table for corporate bookings, but link the booking ID for regular bookings. This behavior was kept intact.
3. **Reschedules and Waitlists:** Rescheduling a booking does not trigger a waitlist promotion for the original class if the member is leaving it.
4. **Error Shapes:** Reschedule validation calls return `{ valid: false, reason }` values, while direct mutations throw `TRPCError` instances.
5. **Unlimited Plan Constant:** Any membership plan offering 999 or more credits is identified as "unlimited" and does not deduct credits upon booking.

Verification and Safety Net

To guarantee correctness, the following commands check the build, types, and logic:

- `<font face="Courier" size="9" color="#b30000">npm run test</font>` verifies the business policies.
- `<font face="Courier" size="9" color="#b30000">npx tsc --noEmit</font>` verifies strict TypeScript typing.
- `<font face="Courier" size="9" color="#b30000">npm run build</font>` compiles and packages the Next.js bundle, validating route definitions.